ChatGPT

认知架构与LLM Agent的再相逢

近年来，大型语言模型驱动的智能体（LLM Agents）蓬勃发展，人们惊讶地发现许多**经典认知架构**中提出的机制正在这些新型Agent中"重现"，有的甚至取得了显著效果。这并非刻意附会：现代Agent在解决复杂任务时遇到的**记忆管理、决策循环、子目标分解**等问题，早在Soar、ACT-R等认知架构中已有深刻研究。因此，重新审视认知架构对当代LLM Agent架构的启发，能为我们提供机制层面的线索，而不仅是口号式的类比。本文系统梳理Soar、ACT-R、LIDA/GWT、BDI等经典架构中的核心机制如何在ReAct、Reflexion、AutoGPT、AutoGen、LangGraph、Voyager、SayCan、ViperGPT、MemGPT、SWE-Agent、OpenDevin、GraphRAG等主流LLM Agent中得到继承或映射，并指出哪些是真正的"认知式"迁移、哪些仅是表面相似。同时，我们讨论这些机制与当前训练范式（SFT、RLHF等）的对应关系，以及未来尚未充分借鉴的经典思想。最后提出一些可验证这些机制作用的实验思路，并给出关键的**takeaways**。

决策循环：从Soar/BDI到ReAct/AutoGPT

经典认知架构Soar提出了**感知-决策-动作**的循环：智能体在每个决策周期观察状态，用**产生式规则**提议可选操作并计算偏好，选择最佳操作执行；若无规则适用则产生**僵局（impasse）**并启动子目标 [1] [2]。BDI（信念-愿望-意图）架构则强调从环境感知Belief、根据Desire产生计划并确立Intention执行，期间只有在一定条件下才放弃当前意图重新规划。现代LLM Agents并未明确引用Soar或BDI，但其**决策循环**与之不谋而合：

- **ReAct**（Reason+Act）是最早将LLM推理与动作交替结合的Agent [3]。每循环包含**一轮内部思考**（LLM生成中间推理）然后**执行外部动作**（调用环境API等），再感知新状态进入下轮 [4]。这一固定节奏被指出**对应经典的"观察-决定-行动"模式**，只是**缺少明确的承诺阶段**（没有像BDI那样在一段时间内坚持某一计划，每步都从头决策） [5]。作者并未明说源自Soar，但结构上的高度同构使得ReAct常被视为认知循环的简单实例 [6]。

- **AutoGPT**等"自主Agent"走得更远，引入了任务列表和反馈循环，被分析认为近似BDI模型 [7]。AutoGPT会维护一系列待办子任务（类似**意图队列**），自动生成方案，逐一执行并根据结果调整计划，被描述为"基于*BDI*的多智能体架构"，采用层次规划和*MCTS*搜索意图序列，并通过历史成功率的*RL*机制调整策略 [7]。尽管AutoGPT的开发者并未正式引用BDI理论，这种架构选择体现了与BDI的同工异曲：Belief对应Agent的工作记忆和长程上下文，Desire对应用户目标，Intention则是当前专注执行的任务。可以说AutoGPT是在无意间**复现**了BDI的决策循环（**D3 机制同源**），只是缺少明文的"致敬"。

- 其它代理框架也体现出类似的循环。如BabyAGI明确采用了"**ReAct范式**"的变体，每次迭代都执行推理+行动，并维护一个全局状态以跟踪任务依赖 [8]。又如Microsoft研制的**AutoGen**，支持多个LLM Agent以消息对话方式协作完成任务 [9]。AutoGen让Agent之间通过自然语言对话交流计划、委派子任务，其核心调度仍遵循"决策-行动-反馈"的闭环，只是将循环拓展为多个代理间的交流。这种**多Agent对话**本质上也实现了一种全局的感知-行动循环：每个Agent感知共享对话（全局状态），决策回应或调用工具，再将结果广播回共享工作空间。

小结： 无论是单Agent的ReAct，还是多Agent的AutoGPT/AutoGen，几乎都显现出认知架构中**循环决策**的影子。这些现代Agent大多**没有直接声明**源自Soar或BDI（**无D1直接继承**），但在机制上"殊途同归"地演化出了**感知-思考-**

行动的节奏，属于**D3级**的同构对应。分析者已经明确指出ReAct与经典“ODA环”（Observe-Decide-Act）的高度重合 [5] 。对于追求通用智能的系统而言，这可能是一种不可避免的架构收敛。

子目标与层次分解：Impasse机制的现代隐现

Soar的一大特色是在决策僵局时触发**子目标**：当没有规则可以选定下一操作（或者候选并列无法抉择）时，Soar会创建一个新子目标去寻求额外信息或制定高层方案，待子目标解决后再返回主任务 [2] 。这个**Impasse->Subgoal**机制使Soar能够 **递归分解问题**，逐步形成层次化的解题策略 [1] [10] 。

现代LLM Agent虽然没有Soar那样明确的架构触发点，但**层次分解**思想广泛涌现：

- **AutoGPT/BabyAGI**这类链式代理，会将复杂目标拆解成可管理的子任务列表（Task List），并在循环中动态调整列表。这相当于**人工实现的子目标栈**：Agent在顶层任务遇到困难（相当于impasse）时，会追加新子任务（subgoals）来获取所需信息或前置条件，然后再返回主任务。尽管开发者未引用Soar，但这种“遇阻->列子任务->再执行”的流程正是Soar子目标机制的翻版。

- **LangChain/LangGraph**等框架通过**Graph**或**链式流程**编码了条件分支与循环，可看作对**HTN（分层任务网络）**的支持：开发者预先定义可能的步骤图，Agent据此在运行时选择路径。这类似于将Soar的运算过程静态化为流程图。当Agent走到某节点无法继续（相当于impasse）时，可以根据图结构进入另一个子流程（subgoal）。作者将其称为“graph-based orchestration”，并强调支持**循环和条件** [11] ——这正是子目标机制的显式表示。

- **Voyager**（2023）则充分体现了自主的层次学习。Voyager是Minecraft环境中的一个开放式LLM代理，它具备**自动生成子目标（curriculum）**的能力 [12] 。Voyager持续探索世界，每当遇到新情景或障碍，就会确定一个探索子目标（如“去某地”或“学会某技能”），通过试错学得对应技能代码并存入库中 [13] 。这种自主形成目标序列并不断深化技能的过程，本质上是**递归子目标细化**：大目标=探索整个世界，被划分为子目标=解锁具体科技树节点，进一步细化为操作序列=具体代码实现。Voyager实现了**层次任务分解和技能累积**，很像Soar在高层决策遇阻时自动生成低层子任务，只不过Voyager使用LLM和环境反馈来实现这一点。

需要指出，**现代Agent的子目标机制多为启发式实现**：AutoGPT依赖提示词让模型自行提出下一步计划（但可能反复空转），Voyager通过环境奖励驱动子目标探索。这些都**缺少Soar那样严格的触发条件和知识编译**。因此，目前尚无证据表明某个LLM Agent **直接继承（D1）**了Soar的impasse机制。但**结构上的相似（D3）**非常明显：将复杂问题分而治之、在需要时引入新子任务，已成为提高Agent可靠性的通用策略。这提示未来可以更系统地借鉴认知架构的子目标框架，让LLM Agent的层次规划和中断恢复更加健壮可控。

工作记忆与缓冲区：ACT-R Buffers vs. LLM上下文

ACT-R认知架构以**模块化缓冲区（buffer）**著称：不同认知模块（如视觉、回忆、目标）各有一个缓冲区，与**中央生产系统**交互 [14] 。每个缓冲限定只能容纳一个“块”（chunk）的信息，如当前目标、检索到的记忆条目等。生产规则根据缓冲内容匹配条件并触发动作，从而更新缓冲或发出行为。通过缓冲区，ACT-R实现了对**工作记忆**的精细建模：有限容量、受注意力控制、模块隔离。

LLM本身并无显式缓冲区——它通过大上下文窗口一次性处理大量文本。然而，在构造LLM Agent时，开发者实际上**分出了类似缓冲的结构**：

- **上下文窗口作为工作记忆：** 在一个Agent决策循环中，给LLM的Prompt往往包含多个部分：指令、最近对话、重要事实、工具输出等等。这可以类比为不同类型信息填入不同"槽"。例如，LangChain设计了**Memory**接口来维护对话历史，或临时信息插入prompt，实现类似工作记忆缓冲的功能。Yao等提出的CoALA框架明确将**Working Memory**定义为一个在LLM调用间持久的数据结构，存储当前决策周期所需的活动变量 [15] [16]。每轮调用会从工作记忆取出相关内容组合prompt，LLM输出再解析回写工作记忆 [16]。这和ACT-R的缓冲读写循环如出一辙，只是信息格式换成了自然语言文本。

- **缓冲容量与门控：** ACT-R缓冲的容量限制意味着认知系统必须对**当前关注什么**做出抉择，否则过载。类似地，LLM Agent的prompt长度有限（上下文窗口约定了最大token），这迫使Agent对**哪部分记忆放入prompt**进行取舍（门控）。许多Agent实现了**检索-填充机制**：在每轮对话，Agent从长期存储（如向量数据库）**检索最相关片段**插入prompt，而不是无区别地堆入全部历史。这相当于对工作记忆缓冲的**选择性填充**，类似ACT-R按照激活度检索记忆 [17]。比如**GraphRAG**采用知识图谱增强检索，通过图结构找到与当前query关联的节点来作为提示 [18]。这保证了工作记忆中存放的是**情境相关**的信息，提高了检索精度——类似ACT-R根据语义激活度提取关联知识 [17]。

- **多层次记忆架构：** 一些Agent更明确地区分快速缓冲和慢速存储，仿佛实现了计算机体系结构中的缓存/内存层次。**MemGPT**便是典型：它受到操作系统**虚拟内存**启发，引入"主上下文"（Main Context，相当于RAM）与"外部上下文"（External Context，相当于硬盘）的分层 [19]。LLM的有限窗口当作主存，额外配置长期存储来分页交换信息，从而给予Agent"近乎无限"的记忆幻觉 [20] [21]。每当Prompt接近长度上限，MemGPT会自动将部分对话或数据**写入外部存储**（持久内存），在需要时再**读回** [22] [23]。这种机制和经典**分层记忆模型**高度一致，也对应认知上**短期工作记忆**与**长期记忆**的区分（Atkinson-Shiffrin模型） [24]。虽然MemGPT论文将其比喻为操作系统，但在认知视角下，它实质实现了工作记忆的**容量拓展**和**内容管理**：正如人脑会在不需要时"遗忘"细节、在需要时努力回忆，MemGPT自动决定哪些信息留在激活状态，哪些归档等待——相当于一种**缓冲区门控**策略。

总的来说，ACT-R缓冲区理念在LLM Agent中有所投射（**D3机制同源**）：**有限且结构化的工作记忆**正成为设计准则。从CoALA的持久工作存储 [15] 到MemGPT的分级存储 [19]，再到LangChain的记忆组件，无不聚焦于**怎样存取和更新Agent的短期信息**。这一切都呼应着认知心理学对工作记忆的认识（Baddeley的多组件模型等）。值得注意的是，尚无LLM Agent作者直接声明"借鉴了ACT-R缓冲"（**无D1证据**），但诸多论文的言语暗示了这种联系。如有研究总结："LangChain 等框架通过链、工具和检索策略构成应用的 **认知架构**" [25] ——至少在社区认知中，这些机制已经被视作某种"认知式"设计了。

长期记忆：情节、语义与程序性在Agent中的演化

人类和认知架构将**长期记忆**划分为不同类型：**情节记忆**（episodic）储存亲身经历事件，**语义记忆**储存一般知识，**程序性记忆**储存技能和规则。经典架构如Soar、ACT-R也分别实现了这些：Soar有情节记忆模块记录状态序列 [26]，语义内存保存事实知识 [24]，ACT-R将程序性知识编码为产生式规则且能学习更新 [27]。现代LLM Agents虽然起初多依赖LLM参数中的**隐性长程记忆**（预训练知识），但很快发展出了**外部记忆体系**来扩展和更新知识库，且有迹象表明对应着情节/语义/程序性三分：

- **情节记忆（Episodic Memory）：** 指智能体对过去经历具体情境的记忆。在LLM Agent中，这往往以**对话日志**或**事件存档**形式出现。典型例子是**Reflexion**：它将Agent每次尝试任务的结果和自我反思存入**情节记**

忆缓冲，下次遇到类似情境时参考 [28]。Reflexion作者将之称为"**将反思文本维护在情节记忆中，以诱导后续更好的决策**" [28]，直言不讳地使用了*episodic memory buffer*的概念。这实际上等价于给Agent增添了一种"一步一记"的能力：如同人会记住"上次我试错时犯了哪些错误"，Reflexion代理在多轮交互中逐渐累积**带时间顺序**的经验库，并在每轮决策前自动检索这些经验总结作为额外提示 [29]。类似地，OpenAI的**Generative Agents**研究也构建了"记忆流"，存储模拟人物每天经历的事件细节，然后通过提取相关记忆塑造角色行为。这些都明确地在模拟情节记忆。此外，一些多模态Agent（如有限的交互环境）会保存传感器数据序列，以备将来回溯——也属episodic性质。

- **语义记忆（Semantic Memory）：** 指一般知识和事实的长期储备。LLM通过预训练已拥有海量世界知识，但Agent往往需要定制或扩展知识，例如连入专有数据库、文档库等。**Retrieval-Augmented Generation（RAG）**是标志性机制：Agent在生成答案时从外部知识源中检索相关条目提供给LLM。这实际构建了一个**显性语义记忆库**。比如**GraphRAG**进一步将知识组织为**图谱**，使Agent能利用实体关系检索信息 [18]——这相当于语义记忆的结构化形式，类比人类的语义网络。认知架构Soar/ACT-R里，语义记忆往往是通过**关联检索**实现的，如根据线索搜索储存的事实 [17]。GraphRAG恰是用节点关系增强了线索匹配，达到了类似效果：实验显示GraphRAG通过多跳关系显著提升了复杂问题答案的可解释性 [30]。又比如**LangChain**允许将资料文档作为知识库，Agent问答时检索片段作为上下文，这也是在构建人工的语义记忆模块。可以说，当代Agent已经普遍具备**语义型长期记忆**：无论通过向量检索、知识图谱，还是嵌入数据库，都是在让LLM以外的地方存放和获取世界知识，以弥补单次对话记忆和参数固化知识的不足 [31] [32]。

- **程序性记忆（Procedural Memory）：** 指如何做事的技能、规则。LLM本身擅长生成步骤描述，但不具备可持久调用的技能库。为提升复杂任务效率，研究者开始显式存储Agent**学到的技能**。**Voyager**提供了一个精彩范例：它在探索Minecraft过程中，将每次成功的行为策略保存为**可执行代码片段**，汇集成不断扩展的**技能库** [12] [13]。这些技能可以在未来被复用，甚至组合来解决新问题 [33]。作者报告："Voyager学到的技能是可延展、可解释且可组合的，这使其能力迅速累积，并减轻了遗忘" [13]。事实上，这正对应Soar/ACT-R的**"知识编译"**概念：将多步推理的成果凝练成新规则存入程序性记忆，下次遇到类似情景直接应用 [34]。Soar称之为**chunking**（块收束） [35]、ACT-R称为**产生式合并**，目的是**避免重复低效的推理** [34]。Voyager把LLM试出来的代码当作chunk留存，性能因此远超"一次性"Agent：取得了比之前方法多3.3倍的物品、行进距离2.3倍、达成科技树里程碑快15倍等成果 [36]。这相当于证明了**程序性知识累积**对长期任务的价值。再看**Reflexion**，虽然没有明确存储规则库，但它的"反思笔记"其实扮演了**准则**的角色——例如经过反思，Agent可能得出一条经验"下次不要忘记检查X"。这些经验写入记忆，相当于新的行为规则被添加，只是形式上是语言而非二进制代码。然而在下轮执行时，LLM会把这句经验当作提示，自然地遵循它行动。可以认为Reflexion通过语言反馈实现了**策略更新**，类似ACT-R为每条产生式维护一个**效用值**，根据奖励更新它以影响将来选用 [27]。不同在于Reflexion不是用数值效用选择规则，而是用自然语言经验直接改变下一次输出内容 [29]。但效果类似——让**过去有用的做法更可能再次出现**。值得一提的是，一些工作正在探索**直接从LLM推理日志提炼程序**：如Zhu & Reid (2023)尝试将LLM Chain-of-Thought转译为Soar产生式规则 [37]。这正桥接了LLM的推理能力和认知架构的可执行规则库，被视为**将LLM"即兴"思考编译成持久知识**的前景方向。

综上，LLM Agents正逐步拥抱**多模态的长期记忆**：既记**经历**又存**知识**还积**技能**。在我们考察的Agent中，**Reflexion**最显式地区分了情节记忆（反思日志）和程序改进（语言提示强化），**MemGPT/GraphRAG**分别体现了对话记忆扩展和语义知识存储，**Voyager**成功展示了技能学习的惊人威力。这些都与经典认知架构的内存体系高度契合。不过需要强调，大部分作者**并未直接声明**"本方法源自某认知理论"（**无D1**），有些只是用类似术语（episodic memory等）做类比（**D2 致敬**），更多的是架构层面的自主演化（**D3 机制同源**）。无论如何，**记忆系统的显性化**是LLM Agent迈向更强能力的共同趋势——这恰恰印证了认知科学的一个基本共识：**智能离不开对过去经验和知识的积累与调用**。

反思与自适应：从RL偏好到语言自我监督

早期的LLM Agent多为一次性策略，**不会从错误中学习**，这与人类和认知模型的行为相去甚远。认知架构提供了两种范式的借鉴：一是**数值型的强化学习**（如Soar-RL给每个操作附加偏好值，根据奖惩调整[38]），二是**符号型的自我反思**（如Soar的回顾模块、ACT-R让模型基于规则成功率调整"期望效用"）。现代Agent在这两方面都开始尝试：

- **语言反思（Linguistic Reflexion）：** 由Reflexion框架开创的做法，让Agent在每轮任务后根据反馈用**自然语言总结**："我哪里做对/错，下次该如何"。这种文字总结被加入下次Prompt，指导模型改进决策[29]。Reflexion作者称之为一种"语言增强的强化（*verbal reinforcement*）"，因为它不更新模型参数，而通过语言反馈提供了类似梯度的信号[29]。他们指出"这类似于人类的迭代学习——通过反思先前失败来形成改进的方案"[39]。比如在编码任务中，Reflexion代理若失败会生成一句反思："代码未考虑输入X的情况"，下次解题时这句提示会让模型避免重犯。同样在决策游戏中，Agent或许会总结"上次走左失败了，应尝试右侧路线"。经过多次迭代，Agent显著提升成功率：[40]报告Reflexion在长序列决策任务AlfWorld中12次学习后成功率提升22%，在HotPotQA推理题上提升20%，在HumanEval编程题上把Pass@1从基线的0%提高到11%。更惊人的是，在HumanEval基准上Reflexion用GPT-3.5系列模型实现了**91%**的Pass@1，超越了GPT-4模型的80%成绩[41]。这表明，仅靠**语言层面的自我反馈**，就能赋予LLM一定的**少样本自适应**能力——一种近似于人类反思学习的本领。

- **策略偏好与RL微调：** 另一方面，一些Agent引入了**强化学习（RL）**或偏好优化来调整决策策略，属于数值方式的自适应。例如AutoGPT据称在架构上融入了根据历史成功调整行动选择的机制[7]。OpenAI早些时候的WebGPT项目为训练浏览网页的LLM Agent，使用了人类反馈强化学习（RLHF）调教其点击和停止的倾向。这些都类似于认知架构里给动作分配"价值"并反复训练（例如ACT-R会用奖励信号更新规则的效用值，在冲突时选择期望效用更高的规则[27]）。不过，与Reflexion的即席自我监督不同，RL类方法往往需要**预先训练**，在线应用中代价高昂，因而当前LLM Agents更多停留在**离线**用RLHF或DPO调优基础模型阶段，运行时则不再学习。这跟Soar-RL的用法倒有几分类似：Soar-RL也是在仿真环境中反复试练规则偏好[38]，一旦投入应用，就使用学到的偏好值而不继续更新。可以说，RL为Agent提供了**数值型偏好记忆**，Reflexion式反思则提供**符号型经验记忆**，两者异曲同工地实现了**让智能体从过去表现中变得更"聪明"**的目标。

- **元认知与可靠性：** 还有一些工作把"反思"机制上升到元认知层面，超越简单的奖励优化。例如**Self-correction**风潮中，谷歌等提出让LLM在回答后自己检查并修改答案（Self-Refine），或让代理生成对自己行为的评价（如"我刚才这一步是否合理？"）。这些做法与Reflexion思路一致，都是赋予模型一定程度的**自省**能力，从而**逐步修正策略**。虽然目前缺乏统一理论指导，但它们隐隐呼应了心理学关于**元认知**的研究（会思考自己思考的能力）。在认知架构中，也有类似概念：例如Soar的**retrospection**模块可以回顾决策过程并总结新规则[42]。有研究者直接将Reflexion称为"ReAct的扩展，类似于Soar的retrospection机制"[42]。可见学界已注意到这种反思环路的认知意味，并认为值得深入探讨其对**Agent鲁棒性和纠错能力**的影响。

总的来说，**反思式学习**为LLM Agent带来了质的飞跃。它**不依赖梯度调整**模型参数，而是通过语言或符号途径让Agent"自己总结教训"，体现了一种**认知风格的强化**。Reflexion是明确受**人类认知启发（D2）**的——作者多次类比人类few-shot学技能[39]。相比之下，纯RL方式与传统AI接轨更多，但其作用（更新策略倾向）与认知架构的机制仍对应（D3）：Soar/ACT-R早已将规则强度、动作偏好数值化，现代Agent无非借助大模型强大的自生成能力，用**语言奖励**或**人类偏好**来实现类似效果。两种范式可以并行：未来Agent或许能在执行任务时**一边用语言反思**，一边**记录数值反馈**，双管齐下地优化决策策略。这将更接近人类学习中显性的反思和隐性的强化同时发生的状态。

多智能体协作：全局工作空间的雏形

当一个任务过于复杂，单个LLM往往力不从心。这催生了**多Agent协作**范式：通过不同角色的LLM分工合作来完成目标。这种架构让人联想到**全球工作空间理论（GWT）**和**黑板系统**：大脑由众多专门化过程组成，通过一个共享的全局工作空间进行通信，获胜的信息经注意选择后**广播**，从而引导整体行为。我们发现，多Agent系统中已经出现了类似**"全局广播+竞争"**的模式：

- **共享记忆/对话即工作空间：** 在AutoGen等框架中，引入了**消息通信**机制，多个Agent以对话形式在同一频道交流 [43]。这个共享对话记录就类似一个**公共黑板**或工作空间，所有Agent都能读取。Adam Hutton将其比喻为"共享'工作记忆'，模块（子*Agent*）将重要信息发布其中供他者获取" [44]。例如一个规划Agent把子任务写出，执行Agent从对话中读到并付诸行动，环境Agent又将结果写回对话供规划Agent注意。这和GWT中的循环极其神似：**专长不同的"心智模块"通过公共工作记忆沟通** [44]。Hutton建议开发者可用一个共享内存总线或事件总线，让各Agent发布目标、中间结果和状态更新，按需读取 [45]。这正是在软件上实现"大脑的全球工作空间"。

- **注意与广播：** 在人脑GWT模型中，众多模块并行自主运行，但只有一个模块的信息可被注意系统选中，进入全局广播，使得其他模块都能获得该信息 [46]。对应到LLM Agents，多Agent协作常需要一种**调度/仲裁**机制，以决定何时由哪个Agent输出信息。比如HuggingGPT中由一个调度Agent根据任务类型决定调用哪个专家模型；AutoGen的设计中也支持人为定义哪个Agent先说、如何回应 [47]。可以将**调度者**视为类比**注意机制**：决定将哪个Agent的产出广播出去供大家共享。某些框架甚至引入**Moderator**角色监视对话，防止Agent陷入无效循环——这类似**注意力抑制无关信息**的功能 [46] [48]。虽然当前实现多为工程heuristic，例如简单地轮流发言或预先固定顺序，但未来或可借鉴GWT，引入根据内容价值动态选择广播的智能调度，使多Agent协作更加高效。

- **竞争与分工：** GWT强调不同认知过程竞争进入全球工作空间。一些Agent系统体现了**角色竞争**或**提案竞争**思想。例如ChatGPT Plugins时代，有工作让多个Agent各自提不同解决方案，再由主Agent评估采用，这里多个方案提供者在**竞争认知舞台**。又比如开发者在设计多Agent时，需决定是一个大Agent包含所有能力，还是多个小Agent各司其职 [49] [50]。经验表明任务划分过细会导致协调困难（Agent互不知彼，输出冲突） [51]；划分过粗则单Agent上下文负担过重。这与认知科学对**模块化 vs 整合**的思考一致：需要**适当数量**的专门模块并通过有效通信协调。正如Hutton用**厨房团队**和**交响乐团**来类比——厨师各有分工但通过喊单（广播订单）和主厨协调，乐团每个乐手自治但靠总谱（共享计划）和指挥节奏（中央协调）来同步 [44] [52] [53]。这些比喻生动地说明了：**没有全局交流的多主体是"一盘散沙"**，而没有专长划分的单主体又难以驾驭复杂任务。因此，多Agent架构需要一个**中心舞台**让信息汇聚，并配合一定的**选择/抑制**机制。现有框架已开始采用**共享记忆池+简单调度**的雏形，正朝认知上的"全局工作空间"靠拢。

总的来看，多智能体LLM系统自发地走向了**GWT式的信息交流模式**。虽然暂无论文直接把GWT挂在嘴边（**无D1明示**），但社区讨论和实践已在**有意无意地致敬（D2）**这一理论 [50]。值得期待的是未来引入更智能的**"选择-广播"循环** [54]：例如通过一个专门的控制Agent，对各Agent输出进行**评分**（基于任务进展、信息增益等）来决定是否广播，全局记忆也可增加标记辅助注意。更远的或许还有**意识/自我模型**的探讨，但就当前而言，即便简单的共享对话+协调，也已提升了复杂任务完成度。例如AutoGen演示了多个Agent合作解数学题、编程等，比单Agent更可靠 [9] [55]。可以说，"全局工作空间"的思想正在助力LLM Agents实现"1+1>2"的团队智能。

时间与资源约束：认知瓶颈对LLM Agent的启示

经典认知模型不仅关注结构，还强调**时序和资源限制**对智能行为的影响。例如**EPIC**架构模拟人类多任务时各种感知-认知-动作通道的**并行与瓶颈**，成功预测了任务反应时。ACT-R也将时间作为核心维度：如检索时间随激活变化、注意有冲突需要排队等。这些对于设计LLM Agent的**调用步长、并行度和恢复策略**具有重要启发，因为大模型的推理开销和错误积累也形成了类似“认知瓶颈”。

- **思考深度 vs. 成功率：** 人在简单任务上往往**快速反应**即可，但复杂问题需要**慢思考**。LLM Agent的链式推理也是如此：更多的ReAct循环可能带来更详尽的思考，但也**成倍增加**API调用成本和出错机会（每步输出都有出错概率）。研究发现现有LLM推理步数过多时收益递减甚至性能下降（这类似人类过长时间工作变疲劳）。因此工程上常限制Agent的思考轮数。例如ReAct默认**每轮只允许1次思考再行动** [4]，避免了模型陷入冗长推理且控制了费用。AutoGPT也引入用户确认机制限制连续loop，以防止无限循环浪费资源。这可以看作**认知经济性**的体现：在给定代价下寻求最佳表现。认知心理学里的**最佳决策停止规则**也强调在边际效益不高时应停止深入求解。Agent设计者实际在摸索类似规则，只是目前多靠经验而非理论。未来或许可以借鉴**理性分析**框架，根据任务复杂度和模型噪声曲线，计算出**最优推理步长**，指导Agent动态决定“思考多久”再作答。

- **并行与串行：** EPIC指出，人类可以感知-认知-动作部分并行，但**中央决策**往往串行一次只处理一道。LLM目前基本是单线程的：一个模型一次只能生成一个序列。然而在多Agent架构或工具调用场景，有机会引入并行。如AutoGen的异步事件驱动架构，让多个Agent**并发等待**不同工具结果，再汇总决策 [56]。又如OpenDevin平台支持同时运行多个子Agent，各自执行代码/浏览网页 [57] [58]。这种并行执行类似**大脑的多通道处理**——视觉、听觉输入并行，手脚不同动作也可同时进行，但**认知控制**依然需要序列化各任务切换（著名的双任务干扰效应）。LLM Agents的瓶颈在于：即使子任务并行，最终仍要汇合到LLM决策上，而LLM的**上下文窗口**有限制无法无限并发融合。工程上，一些框架采取**分阶段**策略：比如LangGraph用显式节点同步点，让并行支线的结果在某节点汇聚，然后再统一交给LLM处理 [53]。这对应了**同步-广播**的节奏，跟乐队演奏中各段落结束要一起“排版”输出类似 [52]。因此，合理利用并行可以加快环境交互、工具调用等I/O bound步骤，但认知推理部分目前仍不可避免地**串行化**。这与EPIC/ACT-R等预测一致：**外围操作可并行，中央认知有瓶颈**。未来，如果出现支持并行推理的模型（如模块化Transformer），也许Agent能真正“一心多用”。但在那之前，我们可以借鉴人类的**时间分割策略**，例如将任务切成短时块，交替处理不同子任务，以在宏观上实现并行目标（就像人轮流监控多个锅做菜）。

- **实时性与恢复策略：** 在某些应用中，Agent需要考虑**响应时限**或**错误恢复**。比如机器人对话必须在用户等待时间内给出答复，否则体验不佳；软件Agent调用API若长时间无结果要超时重试。这与认知架构中**时间约束**的任务（如驾驶时的紧急反应）可以类比。SayCan作为机器人Agent框架，虽然主要贡献在指令+价值评估，但其设计理念也强调**低延迟**、一步规划执行 [59]。ReAct和SayCan都采用了**逐步决策**而非长计划，就是为了快速得到下一个动作 [60]。这可以理解为在环境需要时，以**次优但及时**的行动取代**最优但缓慢**的计划——人类在紧急情况下也常启用快速直觉（Kahneman的“系统1”）而非深度思考。同样，Agent在超出一定思考时间还未出结果时，或许应有**降级策略**（例如直接调用LLM让其不经链路地产出一个回答，哪怕不完美）。一些论文提到类似思路：如当ReAct推理信心不足时可退回CoT回答 [61]。这相当于在人类决策里，时间不够时用**heuristic**直接给答案。关于错误恢复，Soar允许在子目标失败后回溯尝试其他操作，这一点现代Agent也应借鉴：AutoGPT社区经验是设置一个失败计数或冻结机制，当循环多次无进展时自动停止并请求人类反馈。这种处理虽然朴素，但体现了**对付死循环**的意识。在认知建模中，同样会设计停止规则防止模型无限在无解空间打转。可以预见，随着Agent应用到**实时交互**领域，这类时间与资源管理策略将变得愈发关键，值得用认知理论指导其优化。

总而言之，**时间维度**是目前LLM Agent研究中相对薄弱却十分重要的一环。经典认知架构提醒我们：**智能过程并非无限快、无限长**，恰当地限制与调度反而是成功的关键。现代Agent已经开始碰到这些问题苗头，并以工程方案进行补救。将来，把**认知限度**正式纳入Agent设计，例如通过**认知负荷指标**来动态调节推理深度、通过**资源元管理Agent**监控并发与超时（正如计算机有操作系统调度进程），将有助于打造**高效而稳健**的自治代理。

反思：未竟之处与未来机会

在将经典认知机制迁移到LLM Agent的过程中，也出现了一些**似是而非**或尚未填补的空白，需要我们清醒认识：

- **缺乏直接证据的"传承"：** 很多看似受认知架构启发的Agent，实际上作者本人并未承认直接来源，更多是事后观察到的相似。例如ReAct团队并未引用Soar或ACT-R，其出发点只是结合Chain-of-Thought与动作执行[61]；AutoGPT的作者是一名开发者，其架构主要凭经验设计，没有提及BDI理论，尽管结果与BDI流程极为相似[7]。因此我们应避免过度解读"XX Agent就是认知架构YY的延续"这样的说法。如果没有论文明确声明"based on... cognitive architecture"，就属于**D3机制类比**而非真正的D1继承。例如AutoGPT尽管被博客称为采用BDI模型[7]，但没有官方白皮书证实这一点，只能说它**在功能上吻合BDI**而**无设计上的考据**。类似地，一些宣传文章将BabyAGI、HuggingGPT等挂上"五大人格"、"系统1/系统2"等标签，这更多是类比而非基于证据的设计动机。我们在分析中始终区分了这种**启发式相似**和**有证可查**的传承。

- **未被充分利用的经典机制：** 尽管已有许多交集，仍有**成体系的认知思想尚未在LLM Agent中体现**，这意味着潜在的创新空间：

- **稳定-可塑平衡（ART理论）：** 人脑能持续学习新知识又保持旧知识稳定，Adaptive Resonance Theory提出了网络需在"稳定"与"可塑"间动态平衡。目前LLM Agent大多**要么不学习**（保持基础模型参数不变，只用提示和检索）——稳定性有余可塑性不足；要么粗暴fine-tune模型引入新知识——风险是灾难性遗忘，破坏原有能力。还没有Agent具备**在线不断学习新技能同时不忘记旧技能**的机制。或许可借鉴ART，让Agent维持两个存储区，一块稳定地保存已有高价值知识，另一块接纳新知识并定期检查巩固，从而实现在运行中"越用越聪明"且不丢失原本能力。
- **显性-隐性双通道（CLARION）：** CLARION认知架构将认知分为显性规则和隐性神经网络两层，相互协作。LLM Agent目前基本是**显性处理**（用语言推理和API动作），隐性层主要依赖LLM自身的内在知识分布，但Agent不会区分哪部分决策来自"潜意识"。未来可以考虑Agent内部也划分Fast/Slow路径：**Fast路径**由训练好的小模型或heuristic快速反应常规情况（相当于隐性习惯），**Slow路径**调用LLM进行复杂推理（显性思考），两者通过某种元控制切换。这类似Kahneman"快思慢想"理论和CLARION框架的结合。目前除了SayCan这样把value函数当作下意识评分器外，少有系统性探索。开发**Dual System**风格的Agent或许能在效率与复杂推理间取得平衡。
- **全局工作空间的完整实现：** 虽然多Agent通讯开始体现GWT思想，但尚无架构真正引入**"竞争-广播"循环**。大多实现是所有Agent的话都平等写入共享对话，没有一个仿生的竞争过程。将来可能设计一个**全局仲裁模块**，不同Agent可以并行提议想法，该模块选择一条进入公共工作记忆（对话）发布，使其他Agent据此调整。类似的思想在计算机黑板模型上有实践，但用LLM实现且用于实时对话仍未见。OpenAI等或会推出更复杂的Agent Orchestration SDK，其核心无疑将包含**带注意的消息路由**，这正是GWT可大展身手之处。
- **层级记号与语义指针**：SPA（Semantic Pointer Architecture）通过高维向量来表示组合概念，实现了向量化的符号操作。Transformer某种程度上做了这件事，但Agent开发中还没有人明确利用"语义指针"来在外部表示和操作知识。GraphRAG算一例萌芽，用知识图谱embedding作为向量化表示。未来或许可以设计Agent的记忆向量，让不同概念通过算子组合，进行推理，比如将两个记忆向量卷积以查询其关联。这会把Agent提升到一个**神经符号融合**的新层次。当前的RAG多停留在检索+拼接，不真正操作内容，语义指针思想可以引入更复杂的记忆计算。

- **情境感知与元记忆：** 人类有元记忆知道自己记得什么、不记得什么。Agent目前缺少对自身知识边界和记忆内容的监控，经常产生幻觉。认知架构如Soar曾有元层判断决策是否完备，AI中也有监控模块。为Agent加入**内省模块**，让它在回答前检视所用信息是否充分、逻辑是否自洽，类似**检查全局工作空间的一致性**，这可以减少荒谬输出。在Reflexion中我们看到了初步的自我检查迹象（LLM自评解决方案是否成功[62]），但更通用的**自我监控**机制仍欠缺。

总之，认知架构提供了一座富矿，许多机制尚未被"迁移"到LLM Agent领域。一方面，因为大模型强大的生成能力，让人觉得许多任务prompt拼一拼就行，无需复杂控制；但随着应用需求提高，单靠prompt调度已力不从心。我们预测未来研究将更多**主动对接认知科学**：不只是借概念做类比，而是**实证地引入这些机制**，并通过消融和评测检验其作用。这将开辟LLM Agent性能和可靠性的新空间，也是实现更类似人类智能行为的关键路径。

实验展望：验证认知式机制的影响

要深入理解上述认知机制在LLM Agent中的作用，必须设计**可复现的实验**与**消融研究**。以下是几个建议的方向：

1. **全局广播 vs. 单Agent：** 构造一个需要多种能力协作的任务（如复杂推理+计算+检索）。设置两种Agent：一种是单LLM串行调用工具，另一种是多个专长Agent共享一个对话黑板解决。比较它们在任务成功率、效率上的差异。如果多Agent架构优于单Agent，且随任务复杂度增长优势扩大，即可证明**全局工作空间**式的协作在复杂任务中确有价值。进一步地，引入一个**竞争-选择**机制（例如让两个Agent并行提方案，采用评价高的一方）看能否胜过简单轮流发言。如效果显著，将验证**GWT竞争-广播循环**对性能的提升，并量化其贡献。

2. **有无反思记忆：** 对比Reflexion风格Agent与普通Agent在持续学习场景的表现。例如让Agent多次尝试一个Puzzle或编程题。普通Agent每次重头来过，Reflexion Agent保留历次失败教训。统计成功率随尝试次数的提升曲线。如果反思Agent曲线陡峭得多（正如论文报道Reflexion数次迭代后成功率大增[40]），则证明**情节记忆+自我反馈**机制确实能让Agent高效从错误中学习。还可做消融：Reflexion中去掉存储记忆或去掉反思生成任何一个环节，观察性能下降程度，以定位是"写入记忆"还是"提取记忆"哪个环节更关键，抑或两者相辅相成。

3. **子目标分解效益：** 设计一个需要多步骤才能解决的任务（如分段计算或长代码调试），让Agent一种情况下**一次性思考直接给答案**，另一种情况下**先规划子步骤再逐一完成**。例如，让一个Agent不借助ReAct，直接Chain-of-Thought一大段推理然后输出；另一个Agent用ReAct方式，每步小结并执行，再继续。比较两者**正确率**和**思路连贯性**。我们预计子步骤Agent在长程任务上成功率更高，因为它将问题分解降低了单次推理长度，减小了LLM遗忘和乱逸的风险。这类实验能量化**Soar式子目标机制**的价值。如果可能，也可引入**时间压力变量**：在宽松时间下直接一步完成可能成功率尚可，但在要求一定步数内解出时，子目标法优势会更加明显（类似人类在压力下更需要策略）。

4. **工作记忆容量与干扰：** 探究**缓冲区门控**的重要性。方法是在问答任务中，控制插入prompt的知识干扰量，或调整Agent上下文记忆提取的精准度。比如，让Agent回答问题时附加一些不相干的信息，看是否影响正确率；或者比较在Memory模块中**检索Top3相关文档**和**检索Top10**的效果——后者包含较多无关内容，等于工作记忆掺杂噪音。若发现无关信息多时，Agent回答准确率明显下降，说明**工作记忆受干扰**会妨碍决策。这验证了**有限缓冲+注意过滤**的必要性，印证ACT-R缓冲设计的意义。进一步的，可以尝试优化策略（如引入简单的**注意评分**剔除无关内容）是否能恢复性能，从而证明在Agent中实现**注意式门控**的价值。

5. **实时交互与响应延迟：** 评估**认知时间约束**。搭建一个模拟实时环境（例如要求Agent在一定步数/时间内完成决策，否则算失败）。比较不同策略Agent的任务完成率与平均用时。例如，一个Agent深度思考规划

（如AutoGPT长链），一个Agent粗粒度规划快速出手（如SayCan单步决策[59]）。预期在严格时限下，快速决策Agent反而完成更多任务，因为深度规划者常超时。这个实验将揭示**快思考 vs. 慢思考**的权衡，并帮助确定在不同任务复杂度和时限组合下，哪种策略占优。这样的结果可以指导Agent在运行时动态调整策略——这正是认知科学中**时间压力对认知策略切换**的课题在机器智能上的反映。

通过上述实验，我们不仅能验证哪些认知架构机制对Agent性能有显著影响，更能量化影响程度，发现不同机制间的交互作用。例如，全局广播+子目标是否比单用其一效果更佳？反思记忆是否在高噪音工作记忆下仍有效？这些问题的答案将推动我们设计出更"聪明"的Agent，同时也为认知理论在人工智能中的应用提供新的佐证。

结语和Takeaways

回顾以上，我们可以提炼出几点重要认识：

1. **经典认知机制正在Agent中"重生"：** 大量LLM Agent结构无意间复现了认知架构思想——ReAct的**推理-行动交替**对应**认知循环**[4]、Reflexion的**语言反思+情节记忆**呼应**人类元认知学习**[29][39]、多Agent对话共享内存仿佛**全局工作空间**[44]、Voyager积累技能如同**产生式系统编译知识**[13]。这些同构并非巧合，说明认知问题的普适解法在人工系统中也会涌现。

2. **直接传承少，灵感致敬多（D1少D2/D3多）：** 尽管机制相似，**鲜有作者直接声明**"基于某认知架构"设计Agent（例如没有论文说"我们基于Soar实现LLM Agent"），因此严格的D1案例罕见。多数情况是作者**明确致敬认知概念（D2）**，如Reflexion承认模仿人类反思式强化[39]；或者纯属机制上相符（D3），由后来者指认，如AutoGPT被视作BDI模式[7]。我们应据证区分：有证据时大胆连接，证据不足时注明类似之处但不夸大。当前来看，LLM Agent领域和认知科学尚处于**若即若离**状态，存在巨大合作潜力。

3. **认知式改造显著提升了Agent能力：** 已经引入的认知机制普遍**改善了Agent的鲁棒性和任务完成度**。Reflexion通过episodic反思让编码等任务成功率大涨[41]；Voyager凭借技能记忆实现长期自主学习，在Minecraft取得压倒性优势[36]；MemGPT扩充工作记忆后能处理长文档和长对话[63][20]；SWE-Agent定制接口（某种工具工作记忆）使代码Agent性能超越以往模型[64]。这些实例证明，**把大模型围绕认知结构"武装"起来**，能弥补其固有缺陷（如短记忆、不会学习）并释放更大潜能。认知架构不再只是理论，而是实打实地提供了有效的设计模式。

4. **评测范式需升级以捕捉"真智能"：** 判断Agent是否具备类似人类的认知机制，不能只看一次性任务成功率，而应考察**持续交互、适应和解释**能力。例如，可设计**终身学习基准**：让Agent连贯地完成一系列相关任务，看其是否积累经验改进（测情节记忆作用）；**复杂推理基准**：要求Agent在噪声干扰下多步推理（测工作记忆门控）；**协同基准**：多Agent分工解决难题（测全局广播/竞争效果）；**实时响应基准**：限定决策时间窗口（测认知调度）。通过这些评测，才能区分出真正内含认知式架构的Agent，与那些只是堆叠prompt的"伪Agent"。目前少数工作开始朝这方向努力（如Generative Agents用社会互动长程评测），未来评测标准势必会更加关注**学习曲线、健壮性和长时表现**，推动Agent往更真实的智能进化。

5. **未来方向：让Agent更"思考如人"：** 认知科学提供了丰富蓝图，许多尚未实现：如**持续自主学习**（避免遗忘的机制）、**双系统决策**（快慢结合提高效率与准确度）、**自我模型与自省**（提升可靠性和安全）、**情感和动机模型**（丰富交互表现）等。这些都是下一步可以切入的研究切口。例如，将**稳定-可塑**算法融入Agent记忆，使其边用边学且不忘本事；实现一个**显隐混合**的架构，让Agent既有简单模式处理routine，又能在遇到新问题时调动大模型深思；构建**代理版全球工作空间**，实现真正的并行竞争和注意力调控。通过跨学科合

作，我们有望看到LLM Agents从目前的“工具使用者”升级为更加自主、健壮、类人的认知实体，朝着通用人工智能迈出踏实的一步。

参考文献：

【1】 Hengyuan Hu *et al.* (2024). “Cognitive Architectures for Language Agents (CoALA).” *TMLR*. pp. 9–10. [65] [15]

【2】 Shunyu Yao *et al.* (2022). “ReAct: Synergizing Reasoning and Acting in Language Models.” *arXiv preprint arXiv:2210.03629*. (ICLR 2023) [60]

【3】 Shunyu Yao *et al.* (2023). “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.” *arXiv preprint arXiv:2305.10601*. (Section 4.1 mentions ReAct) [5]

【4】 Noah Shinn *et al.* (2023). “Reflexion: Language Agents with Verbal Reinforcement Learning.” *NeurIPS 2023*. pp. 1–2. [28] [39]

【5】 Sherman Wong (2024). “LLM Agent Studies Chapter 1 – Cognitive Architectures.” *Blog*. (Analogies of cognitive limits in agent design) [44] [46]

【6】 Guanzhi Wang *et al.* (2023). “Voyager: An Open-Ended Embodied Agent with Large Language Models.” *arXiv preprint arXiv:2305.16291*. (v2 Oct 2023) [13] [36]

【7】 Scott Wu (2024). “Introducing Devin, the first AI software engineer.” *Cognition Blog*. (Devin and SWE-Bench) [64]

【8】 Akanksha Sinha (2025). “Agent Orchestration: When to Use LangChain, LangGraph, AutoGen — or Build an Agentic RAG System.” *Medium*. (LangChain architecture & LangGraph) [25] [66]

【9】 *Accessible Learning* (2023). “BabyAGI vs AutoGPT: A Comprehensive Technical Analysis.” (Claims AutoGPT uses BDI model) [8] [7]

【10】 Adam Hutton (2025). “Jumping into Agent Design.” *Medium*. (Global Workspace analogy for multi-agents) [44] [46]

---

[1] [2] [3] [4] [10] [15] [16] [24] [26] [31] [32] [35] [38] [59] [60] [61] [65] arxiv.org
https://arxiv.org/pdf/2309.02427

[5] [6] [14] [17] [27] [34] [37] [42] Applying Cognitive Design Patterns to General LLM Agents
https://arxiv.org/pdf/2505.07087

[7] [8] BabyAGI vs AutoGPT: A Comprehensive Comparison of AI Agent Frameworks | Accessible Learning World
https://www.accessiblelearning.in/babyagi-vs-autogpt-a-comprehensive-comparison-of-ai-agent-frameworks

9 55 [2308.08155] AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation
https://arxiv.org/abs/2308.08155

11 25 43 47 66 Agent Orchestration: When to Use LangChain, LangGraph, AutoGen — or Build an Agentic RAG System | by Akanksha Sinha | Medium
https://medium.com/@akankshasinha247/agent-orchestration-when-to-use-langchain-langgraph-autogen-or-build-an-agentic-rag-system-cc298f785ea4

12 13 33 36 [2305.16291] Voyager: An Open-Ended Embodied Agent with Large Language Models
https://arxiv.org/abs/2305.16291

18 The Future of AI: GraphRAG – A better way to query interlinked ...
https://techcommunity.microsoft.com/blog/azure-ai-foundry-blog/the-future-of-ai-graphrag-%E2%80%93-a-better-way-to-query-interlinked-documents/4287182

19 20 21 22 23 63 [2310.08560] MemGPT: Towards LLMs as Operating Systems
https://ar5iv.labs.arxiv.org/html/2310.08560

28 29 39 40 62 [2303.11366] Reflexion: Language Agents with Verbal Reinforcement Learning
https://ar5iv.labs.arxiv.org/html/2303.11366

30 [PDF] Research on Graph-Retrieval Augmented Generation Based ... - arXiv
https://arxiv.org/pdf/2506.15241

41 [2303.11366] Reflexion: Language Agents with Verbal Reinforcement Learning
https://arxiv.org/abs/2303.11366

44 45 46 48 49 50 51 52 53 Jumping into Agent Design | by Adam Hutton | Medium
https://adamhutton.medium.com/jumping-into-agent-design-56d305ad9150

54 [PDF] arXiv:2410.11407v1 [cs.AI] 15 Oct 2024
https://arxiv.org/pdf/2410.11407

56 AutoGen Might Be the Best Way to Build AI Agents — Deep Dive Part 1
https://ai.plainenglish.io/autogen-might-be-the-best-way-to-build-ai-agents-deep-dive-part-1-b826e6c11249

57 58 Agents might be the next frontier of AI and OpenDevin wants to open source them
https://the-decoder.com/agents-might-be-the-next-frontier-of-ai-and-opendevin-wants-to-open-source-them/

64 NeurIPS Poster SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering
https://neurips.cc/virtual/2024/poster/93753